

Lab 3 110550022

Introduction

This report explains the code and shows the experiment results of lab 3.

Implementation Details

Multi-head Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, dim=768, num_heads=16, attn_drop=0.1):
        super(MultiHeadAttention, self).__init__()
        self.generate_qkv = nn.Linear(dim, 3 * dim)
        self.dropout = nn.Dropout(attn_drop)
        self.num_heads = num_heads
        self.dim = dim
        self.sqrt_d_k = math.sqrt(dim // num_heads)
        self.output_linear = nn.Linear(dim, dim)

    def forward(self, x):
        """ Hint: input x tensor shape is (batch_size, num_image_tokens, dim),
            because the bidirectional transformer first will embed each token to dim dimension,
            and then pass to n_layers of encoders consist of Multi-Head Attention and MLP.
            # of head set 16
            Total d_k , d_v set to 768
            d_k , d_v for one head will be 768//16.
        """
        batch_size, num_toks = x.shape[0], x.shape[1]
        qkv = self.generate_qkv(x)
        qkv = torch.reshape(qkv, (batch_size, num_toks, self.num_heads, -1)).swapaxes(1, 2)

        q, k, v = torch.chunk(qkv, 3, -1)

        QK_T = q.matmul(k.transpose(2, 3)) / self.sqrt_d_k
        SQK_T = nn.functional.softmax(QK_T, dim=-1)
        SQK_T = self.dropout(SQK_T)

        result_weights = SQK_T.matmul(v).swapaxes(1, 2).reshape(batch_size, num_toks, self.dim)

        return self.output_linear(result_weights)
```

Given an input shape of $[b, n, d]$, Q, K and V can be generated in one go as a concatenated tensor with a single linear layer with $3*d$ output channels since the operations are independent of each other. The resulting tensor will be of size $[b, n, 3*d]$.

The qkv tensor is then reshaped following the multi-head architecture. Since d_k (same size as d_q) and d_v are both 16, the entire qkv tensor is can be reshaped in one pass, from $[b, n, 3*d]$ to $[b, n, h, 3*d / h] = [b, n, h, 3*d_k]$. The middle 2 dimensions are swapped $[b, h, n, 3*d_k]$ so that the token dim dimensions are in

consecutive order. This way scaled dot product can be applied the same way as in single-head self-attention where the head dimension is not present.

The q, k, v tensors are split from the concatenated tensor along the dim axis, each having the size [b, h, n, d_k]. Scaled dot product is calculated by multiplying q and kT, where kT is k transposed in the n and d_k dimensions ([b, h, d_k, n]), as b and h represent multiple copies of scaled dot product operations. The resulting tensor is divided by the normalizing term $\sqrt{d_k}$ and has size [b, h, n, n].

Next the tensor is passed through a softmax function along the last dimension, ensuring that each row sums up to 1, and the attention dropout layer is applied right after. The resulting tensor is multiplied by v to produce the output of the attention formula ([b, h, n, n] * [b, h, n, d_k] = [b, h, n, d_k]).

The attention output of each head is concatenated through a reversed process of the operations performed earlier.

$[b, h, n, d_k] \Rightarrow [b, n, h, d_k] \Rightarrow [b, n, h * d_k] = [b, n, d]$

This tensor is finally passed through the output linear layer of the multi-head attention module.

Training the bidirectional transformer

```
##TOD02 step1-1: input x fed to vqgan encoder to get the latent and zq
@torch.no_grad()
def encode_to_z(self, x):
    codebook_mapping, codebook_indices, _ = self.vqgan.encode(x)
    codebook_indices = codebook_indices.view(codebook_mapping.shape[0], -1)
    return codebook_mapping, codebook_indices
```

The description wants this function to return latent and Zq given an input. Zq is Z that is quantized through the codebook and latent is the indices of the chosen codebook entries. These 2 arrays can be easily obtained via calling the encode function in the VQGAN class. The codebook indices output by the function, however, has the entire batch merged into a large 1D tensor, so view is called to reshape the tensor into [b, latent_dim ** 2].

```

def gamma_func(self, mode="cosine"):
    """Generates a mask rate by scheduling mask functions R.

    Given a ratio in [0, 1), we generate a masking ratio from (0, 1].
    During training, the input ratio is uniformly sampled;
    during inference, the input ratio is based on the step number divided by the total iteration number: t/T.
    Based on experiments, we find that masking more in training helps.

    ratio: The uniformly sampled ratio [0, 1) as input.
    Returns: The mask rate (float).

    """
    if mode == "linear":
        return lambda ratio : 1 - ratio
    elif mode == "cosine":
        return lambda ratio : np.cos(np.pi * ratio * 0.5)
    elif mode == "square":
        return lambda ratio : 1 - ratio ** 2
    else:
        raise NotImplementedError

```

Not much to say about this part.

```

def forward(self, x):
    _, z_indices = self.encode_to_z(x) #ground truth

    mask_ratio = self.gamma(np.random.uniform())
    # mask = torch.rand_like(z_indices) < mask_ratio
    # masked_z_indices = z_indices.clone()
    # masked_z_indices[mask] = self.mask_token_id
    num_mask_tokens = math.ceil(mask_ratio * z_indices.shape[1])

    mask_positions = torch.rand(z_indices.shape, device=x.device).topk(
        num_mask_tokens,
        dim=1
    ).indices

    mask = torch.zeros(z_indices.shape, dtype=torch.bool, device=x.device)
    mask.scatter_(dim=1, index=mask_positions, value=True)

    masked_z_indices = torch.where(mask, self.mask_token_id, z_indices)

    logits = self.transformer(masked_z_indices) #transformer predict the probability of tokens
    return logits, z_indices, mask

```

This section follows section 3.1 in the paper. The tokens are first obtained through the `encode_to_z()` function, and the mask ratio is calculated by applying the gamma function to a randomly sampled r . Then $\text{ceil}(\text{gamma}(r) * N)$ tokens are selected using a combination of `rand` and `topk`. I originally used a probabilistic implementation that selects all indices with a random value lower than the specified mask ratio. However, after implementing the inpainting section, I realized this implementation will have worse stability and went back to the paper to double check the details.

After obtaining the mask, a masked z indices tensor is generated and passed into the transformer for prediction. In addition to the 2 default return values, I've also returned the mask so that calculating the cross entropy of masked tokens only is possible (assuming that I understood the paper correctly).

```

def train_one_epoch(self):
    total_loss = 0.0
    num_batches = 0

    for step, batch in enumerate(tqdm(self.train_loader)):
        batch = batch.to(self.device)
        logits, targets, mask = self.model(batch)
        #print(logits.shape, targets.shape, mask.shape)
        flat_mask = mask.view(-1)
        loss = F.cross_entropy(logits.reshape(-1, logits.size(-1))[flat_mask], targets.reshape(-1)[flat_mask])
        loss.backward()

        if (step + 1) % self.acc_steps == 0:
            self.optim.step()
            self.scheduler.step()
            self.optim.zero_grad()

        total_loss += loss.item()
        num_batches += 1

    if (step + 1) % self.acc_steps != 0:
        self.optim.step()
        self.scheduler.step()
        self.optim.zero_grad()

    avg_loss = total_loss / num_batches if num_batches > 0 else 0.0
    print(f"Training Loss: {avg_loss:.4f}")

    torch.save(self.model.transformer.state_dict(), './checkpoints/last_ckpt.pt')

```

Before calculating the cross entropy between the masked predicted logits and the masked targets, the first 2 dimensions of the logits b and $\text{latent_dim} \times 2$ and the target dimensions are merged into a single dimension of length $b \times \text{latent_dim} \times 2$ to calculate the cross entropy across the entire batch.

The optimizer and scheduler are only updated every few steps, and `zero_grad()` is only called right after. I encountered a bug where I did not remove the `zero_grad()` function that was added before implementing the accumulated steps which made the training only use the gradient of 1 in every `acc_steps` batches.

After the loop, any remaining gradient is used to update the weights.

Image Inpainting

```

with torch.no_grad():
    z, z_indices, _ = self.model.vqgan.encode(image)
    z_indices = z_indices.view(1, -1)
    #z_indices[mask_b] = self.model.mask_token_id #z_indices: masked tokens (b,16*16)
    mask_num = mask_b.sum() #total number of mask token
    z_indices_predict=z_indices
    mask_bc=mask_b
    mask_b=mask_b.to(device=self.device)
    mask_bc=mask_bc.to(device=self.device)

    #raise Exception('T0003 step1-1!')
    ratio = 0
    #iterative decoding for loop design
    #Hint: it's better to save original mask and the updated mask by scheduling separately
    for step in range(self.total_iter):
        if step == self.sweet_spot:
            break
        #ratio = self.model.gamma(i / (self.total_iter - 1)) #this should be updated

        z_indices_predict, mask_bc = self.model.inpainting(z_indices_predict, mask_bc, mask_num, step, self.total_iter)

```

The only changes to `inpainting.py` were the calculation of `z_indices` and the

parameters of the inpainting function. Z_indices is reshaped that way to add an arbitrary batch dimension so that the shape fits the network.

```
def inpainting(self, tokens, mask, total_masked, step, total_iters):
    masked_tokens = tokens.clone()
    masked_tokens[mask] = self.mask_token_id

    logits = self.transformer(masked_tokens)
    #Apply softmax to convert logits into a probability distribution across the last dimension
    #raise NotImplementedError
    probs = torch.softmax(logits, dim=-1)

    #FIND MAX probability for each token value
    z_indices_predict_prob, z_indices_predict = probs.max(dim=-1)

    if step < total_iters - 1:
        ratio = self.gamma(step / (total_iters - 1))
        next_ratio = self.gamma((step + 1) / (total_iters - 1))

        #predicted probabilities add temperature annealing gumbel noise as confidence
        g = -torch.log(-torch.log(torch.rand_like(z_indices_predict_prob) + 1e-10) + 1e-10)
        temperature = self.choice_temperature * (1 - ratio)
        confidence = z_indices_predict_prob + temperature * g

        #hint: If mask is False, the probability should be set to infinity, so that the token is not masked
        #sort the confidence for the rank
        #define how much the iteration remain predicted tokens by mask scheduling
        ##At the end of the decoding process, add back the original(non-masked) token values
        confidence[~mask] = float('inf')

        # Calculate number of tokens to unmask
        #num_masked = mask.sum().item()
        num_to_unmask = mask.shape[1] - int(total_masked * (next_ratio))
        top_confidence_positions = confidence.topk(num_to_unmask, sorted=False).indices
        #print(top_confidence_positions)
        mask_bc = mask.clone()
        mask_bc[0, top_confidence_positions] = False
        z_indices_predict[~mask] = tokens[~mask]
        return z_indices_predict, mask_bc
    else:
        z_indices_predict[~mask] = tokens[~mask]
        return z_indices_predict, mask
    #print(mask_bc)
```

The masked z_indices are calculated and passed into the transformer for prediction, and the output logits are converted into probabilities using the softmax function. The maximum probabilities is then selected as the token prediction.

Next the program tests if it is the last iteration in the decoding process. If it is not, the program will calculate the confidence of each token and work out the number of tokens to unmask in the next iteration. Since the hints says to set the confidence of unmasked tokens to infinity, when selecting the tokens to unmask using topk, the number of already unmasked tokens should also be added to

num_to_unmask. So the calculation of num_to_unmask can be simplified to the total number of tokens – the number of masked tokens in the next iteration.

After calculating the mask for the next iteration, the unmasked tokens are added back to the prediction using the original mask.

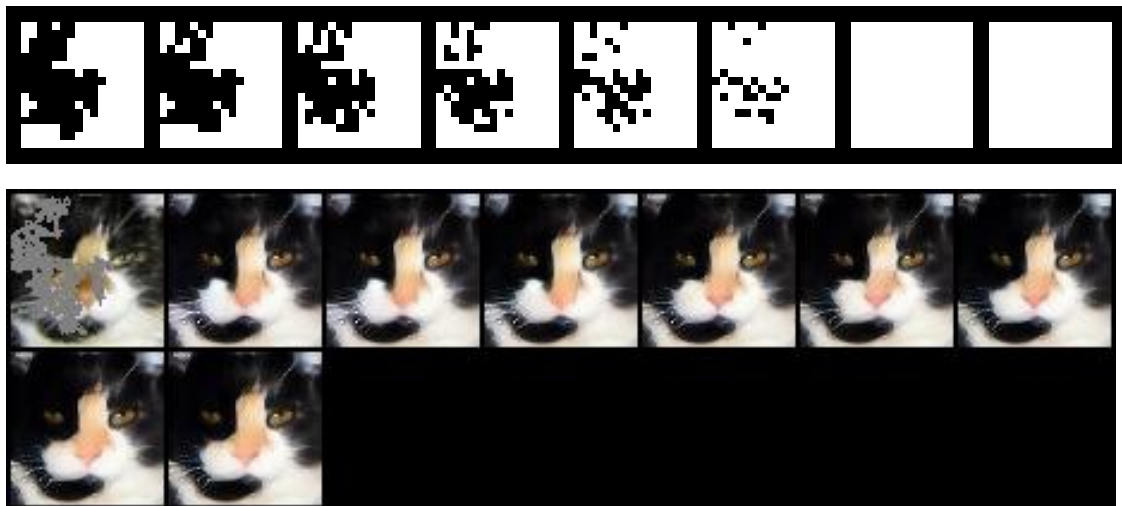
Discussion

During training, the accuracy plateaus after around 50 epochs and the model doesn't seem to improve much afterwards. After testing with the checkpoints, however, the checkpoints at a later epoch (100+) was able to produce better results halfway through the decoding iterations than finishing the process completely, and the model at 50 epochs was able to produce slightly superior results by going through all the iterations.

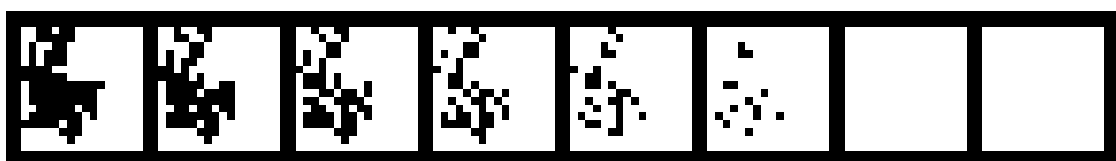
Experiment

Part 1

Cosine

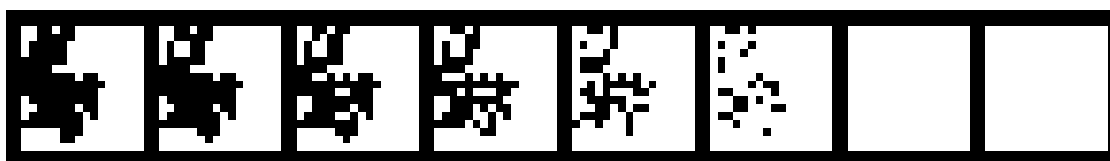


Linear



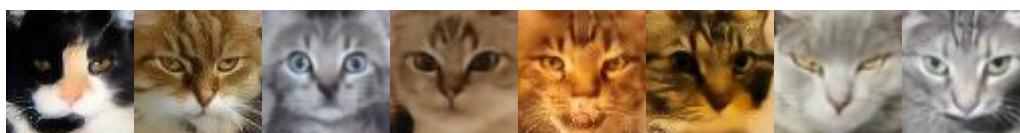
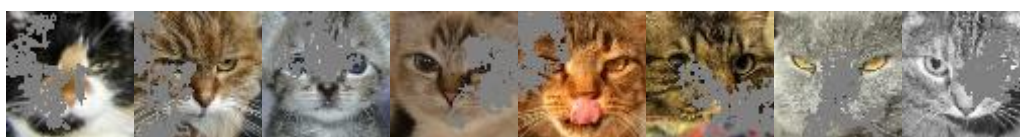


Square



Part 2

```
(dlcourse) PS C:\Users\jonat\Documents\deep_learning_course\lab3> cd .\faster-pytorch-fid\
(dlcourse) PS C:\Users\jonat\Documents\deep_learning_course\lab3\faster-pytorch-fid> python .\fid_score_gpu.py --predict
ed-path ../test_results --device cuda:0 --num-workers 0
747
100%|████████████████████████████████████████████████████████████████████████████████| 15/15 [00:03<00:00, 3.81it/s]
100%|████████████████████████████████████████████████████████████████████████████████| 15/15 [00:01<00:00, 11.44it/s]
FID: 35.405174540602445
```





Hyperparams

```
def configure_optimizers(self):  
    optimizer = torch.optim.AdamW(self.model.transformer.parameters(), lr=1e-4, weight_decay=1e-4)  
    scheduler = get_linear_schedule_with_warmup(optimizer, self.total_steps // 10, self.total_steps)  
    return optimizer, scheduler
```

For the training hyperparameters, I followed some recommendations online on training transformers and chose to use adamw with `get_linear_schedule_with_warmup` from HuggingFace. The scheduler code is short, so it was directly migrated into the same file.

Batch size is set to 20 and trained for around 50 epochs. I've tried a checkpoint at 120 epochs and it doesn't work as well, although it is still able to get an FID score lower than 40 (around 36).

The mask scheduling function is cosine during training and inpainting as it is the recommended setting in the original paper, and all the other settings in `maskgit.yml` was also kept as default.